

Alfred Siby Cyriac
CSCI 355 Exam 1 Study Guide

① JavaScript Basics

• Primitive Data Types:

- 1) Number - all numbers are 64-bit IEEE 754 Floating point. There is no separate int, double, or boolean type
- 2) String - single or double quotes are interchangeable, backticks allow template literals allowing variables to be inserted into strings using `${}`
- 3) Boolean - true or false value
- 4) undefined - value not yet assigned
- 5) null - explicitly set to represent an empty/non-existent reference

* Undefined no value set YET, whereas null means its intentionally set to nothing*

• Variable Declarations:

- 1) Keyword 'let' is scoped to the curly braces in which it was declared and you are able to reassign its value
- 2) Keyword 'const' is scoped to the curly braces in which it was declared, its value can NOT be reassigned and a value must be initialized when declared otherwise it results in an error
- 3) Keyword 'var' is scoped to a function, its value is reassignable
- 4) Declaring a variable with no keyword makes it a global variable

• Functions:

- 1) Function declaration - hoisted to the top of its scope meaning it can be called before it appears in the code
- 2) Function expression - not hoisted meaning the function can only be used after its defined as it is stored in a variable
- 3) Arrow Function: shorthand: `const fn = (x) => x * 2;` they are clean for one-liner functions but anything more it gets complicated

• Objects:

- Anything that is not a primitive is an object
- Objects are key-value pairs denoted by curly braces
- Once an object is defined its properties can be accessed with a dot operator and its values can be changed
- Objects are compared by reference NOT value
- Spread operator `{...obj}` creates a shallow copy

• Array Methods:

- 1) `arr.push(val)` - appends to end
- 2) `arr.indexOf(val)` - returns index or -1 if not found
- 3) `arr.filter(fn)` - returns new array of elements where `fn` returns true (original array unchanged)
- 4) `arr.map(fn)` - returns new array with `fn` applied to every element

• Spread vs Rest Operator

- 1) Spread removes the braces from an array so that it can be used in functions that take comma separated list of #'s
`let arr = [1, 2, 3]`

`console.log(Math.max(...arr));`

- 2) Rest operator is used inside function parameters and the argument becomes an array
function `my-max(...numbers)`

Q: What is the difference between undefined and null in JavaScript?

A: Undefined means a variable has been declared but no value has been assigned YET. null is an explicit assignment representing an empty or non-existent reference.

Q: What is hoisting? Write an example showing var hoisting behavior?

A: Hoisting is the process by which JS moves variable declarations (but not assignments) to the top of their scope before code executes. For example, with `var`: `console.log(x)` prints undefined because the declaration `var x` is hoisted and initialized to undefined. If `x` were declared with `let`, the code would throw a ReferenceError due to the temporal dead zone.

Q: Explain what const means for objects. Does it prevent the object from changing?

A: Const does NOT prevent mutation of object's internal properties. For example, a const object can still have its properties added, changed, or deleted. To make an array truly immutable, you must call `Object.freeze()`.

Q: What is the JS interpreter and compiler?

A: The JS interpreter reads and executes JS code line by line, top to bottom. The compiler on the other hand takes the full code, analyzes it, and produces optimized machine code before execution. In modern JS both the interpreter and compiler run simultaneously with interpreter being much faster but there is a swap that occurs when the compiled version is ready giving us the best of both worlds.

② JavaScript Memory Management

• Call Stack vs Heap

1) Call Stack - stores primitives and function call frames.

Fixed-size, fast access. Each function call adds a stack frame that is removed when the function returns.

2) Heap - Stores all dynamically sized data - objects (including functions and arrays). Variables on the stack hold pointers to heap address.

• Example:

```
let n1 = 31;
```

```
let n2 = n1;
```

```
let f1 = { num: 3, den: 4 };
```

```
let f2 = f1;
```

Call Stack		
Identifier	Address	Value
n1	0x01	31
n2	0x02	31
f1	0x03	0xA1
f2	0x04	0xA1

Heap	
Address	Value
0xA1	{ num: 3, den: 4 }

- Pass by Copy of a Reference:

- Primitives are stored directly on the stack so the assignment of a new variable to be a copy of an existing variable copies the value. This means changes to one variable do not affect the other.
- Objects are stored on the heap. The stack holds a pointer (address) to the heap. Assignment copies the pointer.
- If you copy a pointer and then mutate the object through one variable, both variables see the change due to the shared identical heap address
- If you reassign one variable to a new object, the other variable still points to the old object
- * If you create a new object $\rightarrow$ only one reference changes.
Mutation (.prop =) $\rightarrow$ all references see the change

- Shallow Copy:

- Shallow copies of objects are created using the spread operator.
- Let $f2 = \{...f1\}$ creates a new object, copying each top-level property by value. BUT nested objects inside are still shared by reference

- Scopes:

- 1) Global - declared without keyword. available anywhere including across files (bad practice)
- 2) Function - declared using keyword var and lives inside the function. Hoisted to start of function and initialized as undefined
- 3) Block/lexical - declared using const/let. scoped to the nearest curly braces
- 4) Nested scopes - inner functions can access outer variables, but NOT vice versa

- Closures:

- A closure is a function bundled with references to its lexical environment - the variables in scope at the time the function was created. Closures preserve access to outer-scope variables even after the outer function has returned

Higher-Order Functions:

- A function that either takes a function as input or returns a function as output

- Example:

```
function outer(a) {
  let b = 20;
  let unused = 50;
  return function inner(c) {
    let d = 40;
    return [a, b, c, d];
  }
}
```

```
let j = outer(10);
let j = inner(30);
console.log(j); // outputs: 10, 20, 30, 40
```

Call Stack		
Identifier	Address	Value
outer	0x01	0xA1
i	0x02	0xA2
j	0x06	"10, 20, 30, 40"
New Stack Frame		
a	0x03	10
b	0x04	20
unused	0x05	50
New Stack Frame		
c	0x07	30
d	0x08	40

Heap	
Address	Value
0xA1	function outer(a) { ... }
0xA2	function inner(c) { ... }
	env = { a: 0x03, b: 0x04 }

closure

popped off when the function returns but variables can be preserved by closure

Q: What is the difference between primitive and object assignment in JS memory?

A: Primitives (numbers, strings, booleans) are stored directly on the call stack. Assigning a primitive copies the value meaning changes to one variable does NOT affect the other. Objects are stored on the heap with only a pointer (memory address) on the stack. Assigning an object copies the pointer so changes made by mutating properties applies to both objects.

Q: What is a closure?

A: A closure is a function that retains access to variables from its enclosing scope even after that scope has exited. Very common in higher-order functions with nested functions. It only retains ~~to~~ access to variable it NEEDS.

Q: What is a shallow copy & what is its limitations?

A: A shallow copy creates a new top level object and copies each property by value. The limitation is nested objects as their pointers are copied, not the objects themselves which can cause a lot of confusion. Changes to the nested object would affect BOTH copies. The fix to this is a deep copy / clone.

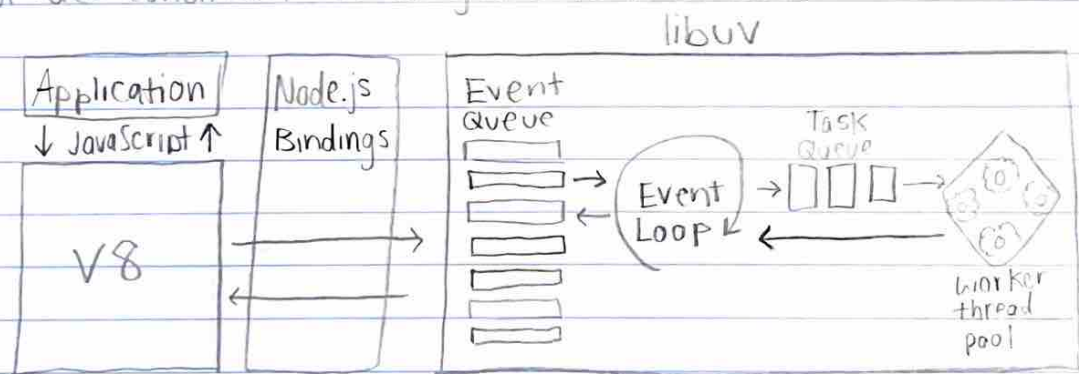
③ Asynchronous Programming

- Why async programming? Procedural (blocking) code executes top-to-bottom and each task must complete before the next begins. When tasks involves I/O (file reads, network requests), they can take unbounded time, freezing the app. Async programming allows the program to continue executing other work while waiting for slow operations.
- Node.js is single-threaded for application code, but uses internal worker threads (via libuv) to handle async I/O without exposing thread complexity to the developer

• Node.js Architecture:

- 1) V8 Engine - Executes JavaScript. Single threaded. Has a call stack (where function calls live) and a heap (where variables are stored). It cannot run two things simultaneously.
- 2) libuv - C library that provides the Event Queue and manages worker threads for async I/O. Polls I/O and queues completed callbacks
- 3) Event Queue - A queue of callback functions that are ready to run. Callbacks are placed here when their async task completes
- 4) Event Loop - Continuously monitors V8's call stack. When the stack is empty, it moves the next callback from the event queue onto the stack for execution

* Existing code on the call stack ALWAYS runs to completion before any callback from the event queue runs. This is not a race condition - it is guaranteed behavior *



- Callbacks: A callback is a function passed as an argument to an async API call. This function specifies what should happen AFTER the async task completes. Node.js APIs follow the convention: `callback(err, data)` - error is always first, data second.
- Continuation Passing Style: To sequence tasks (A then B), put the call to B as the last statement inside A's callback. This guarantees B only runs after A completes.
- Concurrency: To run tasks concurrently, issue multiple async calls one after another without waiting. Since each call is nonblocking, all tasks begin simultaneously. Order of completion is NOT guaranteed.

- The Counting Technique: Use a counter variable that increments each time a callback runs. When `count == total`, all concurrent tasks have finished and you can safely proceed to next step.

- Recursive CPS (N tasks synchronously): To perform N async tasks sequentially: in the call back, check if there are more tasks. If yes, recursively make the next async call. If no, proceed to the final result.

- N Domain Concurrently (Code):

```
for(let i=0; i < domains.length; i++) {
```

```
  resolve(domains[i]);
```

```
}
```

```
function resolve(domain) {
```

```
  dns.resolve(domain, after_resolution) {
```

```
    function after_resolution(err, records) {
```

```
      console.log(domain, records);
```

```
    }
```

```
  }
```

- N Domains Synchronously (Code):

```
let count = 0;
```

```
dns.resolve(domains[count], after_resolution);
```

```
function after_resolution(err, records) {
```

```
  count++;
```

```
  ip_address.push(records);
```

```
  if(count == domains.length) {
```

```
    console.log(ip_address);
```

```
  } else {
```

```
    dns.resolve(domains[count], after_resolution);
```

```
  }
```

```
}
```


Q: Describe the Node.js event loop. What are the roles of the call stack, event queue, and event loop?

A: The call stack in V8 is where function calls are pushed and popped as they execute. Node.js is single threaded so only one function runs at a time. When an async function comes up it's handed off to libuv where the event loop adds it to the task queue. Then worker threads are used to complete the async and once completed places the callback function on the event queue. The event loop checks for when the callstack is empty and when it is it moves the callback from the event queue onto the call stack.

Q: What is continuation passing style? Why is it necessary when chaining async operations?

A: CSP is a pattern where, instead of returning a value normally a function accepts a callback that specifies what to do next. The second task gets called inside the callback of the first task. This ensures task B only begins after task A has completed.

④ OSI & TCP/IP Model

- What is a protocol?

A protocol is a system of rules that allows two or more entities to transmit or receive information. It defines:

- Syntax: the structure/how information is encoded
- Semantics: the meaning/ what is being communicated
- Synchronization: timing and speed of transmission
- Error recovery methods

- Conceptual Models

- Both OSI and TCP/IP are conceptual models built around the notion of layering

- Each of the layers in these 2 models represents an abstraction. Higher layer protocols can assume that lower layer protocols just work by using their provided methods

- Computer protocols are extremely strict and when writing the specifications of one there can be no ambiguity

- The physical layer focuses on how information is encoded. Where that be with:
 - 1) Copper cables → data is encoded as an electric current
 - 2) Optical cables → data is encoded as pulses of light
 - 3) Wireless → data is encoded to radio waves
- The data link layer deals with encoding data from a computing device on to a communication medium and transmitting data from one network device to the next. *Only facilitates transport of data across a single hop*
- The network layer is built on top of multiple data link layers exchanges and provides end to end transport of data. This process of routing traffic across multiple devices is called internetworking.
- Protocol Data Units (PDU): Each layer wraps the previous layer's data by adding its own header.
- Addressing Types:
 - 1) MAC Address - Hardware-level, data link layer. Identifies a specific NIC. 48-bit, globally unique
 - 2) IP Address - Network layer. Identifies a device on a network using IPv4 (32-bit) or IPv6 (128-bit)
 - 3) Logical Port - Transport layer, identifies specific app on device

Q: What is the purpose of layering in the OSI / TCP/IP Model?

A: Layering divides network communication into distinct responsibilities, allowing each layer to focus on a specific function and assume that lower layers just work. This abstraction means higher-layer protocol developers don't need to know whether the physical connection is wired or wireless.

Q: What are protocol data units? How does encapsulation work as data moves down the stack?

A: A PDU is the unit of data associated with a specific layer. As data moves down from the Application layer to Physical layer, each layer adds its own header. At the destination, each layer strips its header as data moves back up.

⑤ Physical Layer

• Twisted Pair Copper Cables:

- Uses voltage modulation to encode bits (charge = 1, no charge = 0)
- A network interface card converts digital bits to/from electrical current
- Crosstalk: interferences caused by magnetic fields induced by current in nearby wires
- Wires are twisted in pairs with opposite polarities → their magnetic fields cancel each other out, minimizing noise

• Fiber Optic Cables:

- Uses light modulation to encode bits through thin glass tubes
- Theoretical max speed bounded by speed of light
- Backbone of the internet; increasingly used for residential connections (Google Fiber, Verizon Fios)

• Channel Types:

- 1) Simplex - Data flows one way. Ex: TV broadcast, baby monitors, security cameras
- 2) Half Duplex: Two-way but parties take turn because cannot send and receive simultaneously. Ex: walkie-talkies, Wi-Fi
- 3) Full Duplex: Two-way simultaneously. Ex: telephone calls, ethernet

• Hubs / Repeaters:

- Physical layer device that copies signal to all ports
- Creates collision domain as only one device can transmit a message at a time
- Two simultaneous transmission = signal corruption for all receivers

• Carrier Sense Multiple Access (CSMA) Algorithm

- Before transmitting, check if the channel is idle. Only transmit if no traffic is detected
- Other variations of this came along but this was the central idea

- Network Switch:

- The switch (a data link layer) replaced hubs in the 1990s. Unlike hubs, switches inspect each frame's destination MAC address and forwards it only to the correct physical port. This eliminates collision domains within a single switch

- All wireless communication (ex wifi) is half-duplex but we can't perceive it

Q: Why are wires in twisted pair cables twisted together?
What problem does this solve?

A: Twisted pair cables solves the problem of electromagnetic interference caused when an electrical current creates a magnetic field that induces noise on nearby cables. Each pair of wires carries equal-and-opposite polarity signals. When twisted together their magnetic fields cancel each other out.

Q: What is the difference between simplex, half-duplex, and full-duplex communication?

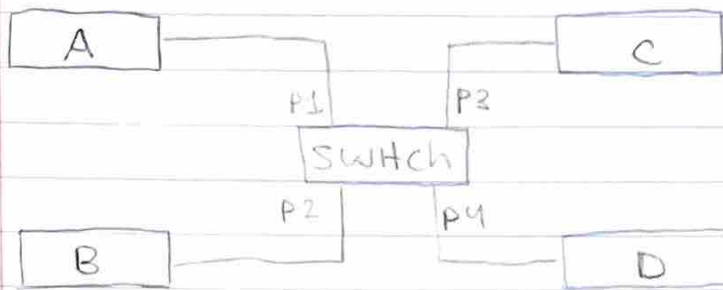
A: Simplex: data flow in one direction, example TV broadcast. Half-duplex: data flows both ways but not simultaneously, example walkie talkie. Full-duplex: data flows both ways simultaneously (phone call).

⑥ Data Link Layer

- The data link layer handles node-to-node transfer of data, that is a single hop from one network device to the next. It also translates from digital packets into analogue.

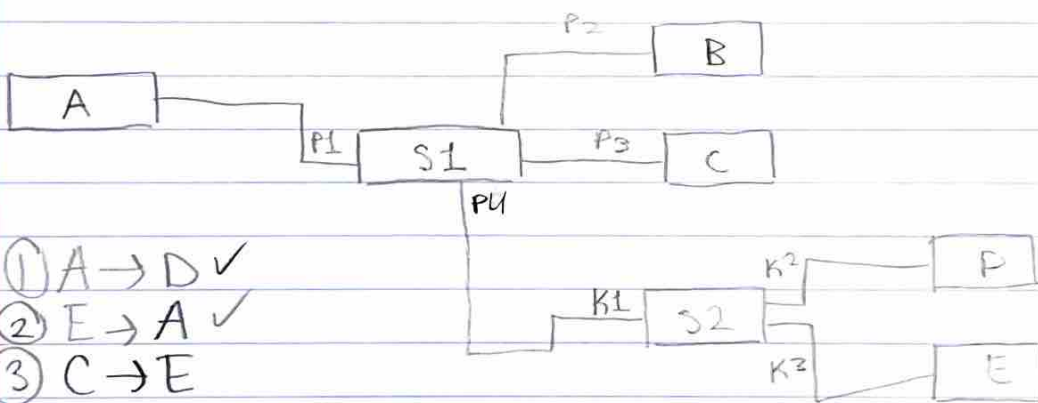
- Why Frames? Frames are wrappers that add meta-information needed to deliver data from one node to the next. The frame header is discarded at each hop since its job is done once the data reaches the next device. A new frame is created for each hop - this keeps frame size constant regardless of total # of hops which is critical for scalability.

- **Frame Control Sequence (FCS):** Provides data integrity with a true/false whether the message has been corrupted or not. It utilizes the CRC32 hashing algorithm and compares its output at point A and B. If they are not the same there 100% was corruption.
- **MAC Address:** A 48-bit, globally unique hardware identifier assigned by manufacturer. The first 24 bits identifies the manufacturer and the last 24 bits is the vendor-assigned portion.
- **Quality of Service (QoS):** Lets the network prioritize time-sensitive traffic (VoIP, gaming, stock trading) over bulk transfers (File downloads)
- As we know network hubs was replaced by network switches and they possessed both CPU and memory. In the case 2 messages are being sent to 1 device simultaneously, it uses the switch memory in best case scenario and in the worst case scenario it uses the priority flag to throw away the less priority device message
- **Switch Device Discovery Algorithm**
 - 1) Upon receiving a message record its source mac, port #, and expiry date into the FIB (Forwarding information base)
 - 2) Check FIB for the destination mac, if it is found then forward it to the mapped port #, else hub mode - forward to all other ports EXCEPT the receiving port
- As each device sends a message, the switch starts to learn/discover the devices making it easier to send messages in the future



- ① A → C ✓
- ② C → D ✓
- ③ D → A ✓
- ④ B → D

MAC	PORT #	Expires
A	p1	x
C	p3	x
D	p4	x
B	p2	x



- ① A → D ✓
- ② E → A ✓
- ③ C → E

S1

MAC	PORT #
A	P1
E	P4
C	P3

S2

MAC	PORT #
A	K1
E	K3
C	K1